

Prosjektbeskrivelse – IT-utviklingsprosjekt (2IMI)

Prosjekttittel

Internet Boardgame Database (IBDb)

Bidragstere

Sivert M. Hansen (Individuelt prosjekt)

1. Prosjektidé og problemstilling

Beskrivelse

- Hva er prosjektet?

Prosjektet en Flask-applikasjon som jeg utvikler for å lære meg python-rammeverket, altså Flask. Det er en skoleoppgave og vi har krav om blant annet å et inloggings-system på plass. Ellers vil jeg utforske funksjoner som søking, visning av database-innhold og brukerdashbord.

- Hvilket problem løser det?

Nettsiden skal la deg som sagt søke opp brettspill og finne info om de. Dette kan man bruke som et verktøy for å researche spillet eller si noe om hvilke målgrupper spillet passer. Da slipper du å komme til spillekvelden med et spill ingen liker!

- Hvorfor er løsningen nyttig?

Nettsiden fungerer som et oppslagsverk for brettspill. Det betyr at du kan søke opp spill du allerede har for kanskje å hvordan det spilles om du f.eks. mistet reglene. Dersom du ikke har et spill kan du bruke nettsiden til å lese deg opp på nett isteden for å måtte dra til byen eller kjøpesenteret.

Målgruppe

Hvem er løsningen laget for?

Løsningen er laget for de som spiller og liker brettspill, og de som vil finne ut mer om de. Det kan f.eks. være før de velger å kjøpe det, eller ikke, p.g.a. det de leste om det. Det kan også f.eks. være de som har mistet regler og glemte hva målet i spillet var som bruker siden for å minne seg selv på dette.

Plan for Prøveksamensdagen

Videre kan du lese om det jeg har planlagt å utføre på selve prøveeksamen.

Lage et ordentlig brukerdashbord

Denne er veldig omfattende så jeg deler den opp i mindre deler:

Endre egen brukerinfo

Hvis brukeren finner ut at de plutselig vil bruke en annen email, et annet brukernavn eller bytte passord, burde de kunne det. I brukerdashbordet vil jeg derfor legge til denne funksjonaliteten.

Gi admin og editor mer kraft

Til nå er den eneste forskjellen på en vanlig bruker og admin/editor muligheten til å registrere nye brettspill. I tillegg er det ingen praktiske forskjeller mellom admin og editor så langt.

Jeg vil at admin skal kunne slette brukere og kunne endre informasjon som tilhører brettspillene. Da slipper jeg å gå inn i databasen for å gjøre dette manuelt.

Annet

Egen side til hvert av brettspillene. Her skal blant annet beskrivelsen komme til bruk, noe den ikke egentlig har vært fra før.

Oversiktlige søkeresultater

Nå hentes bare alt ut av databasen og vises i en parantes, en rå python-liste. Dette er ikke optimalt. For å gjøre det enklest mulig kan jeg ta utgangspunkt i "brettspillkortene" jeg har fra før av.

Lage en side for personvernserklæring/TOS

Sier seg litt selv, bare en som forteller deg hvordan jeg bruker dataen din på nettsiden. Jeg kan da lage en footer med lenke til TOS-en.

At bilder er blobs istedenfor relative paths

Dette gir meg muligheten til at andre enn meg laster opp bilder til databasen. Nå må jeg manuelt legge bilde inn i media-mappen, for å så referere bare til filnavnet når jeg registrerer. Måten jeg har det på nå gjør det umulig for brukere (editor/admin) på nettsiden å legge til bilder ved registrering av brettspill.

3. Teknologivalg

Programmeringsspråk

- Python

Rammeverk

- Flask

Database

- MariaDB

Andre Verktøy

- Waitress (WSGI)
- GitHub

- GitHub Projects (Kanban)

4. Datamodell

Programstruktur

Under ser du en oversikt av de funksjonelle delene som trengs i programmet:

```
boardgame-site/ ├── app.py ├── templates/ | ├── base.html | ├── index.html | ├── register.html |  
├── login.html | ├── dashboard.html | ├── register_boardgame.html | ├── results.html ├── static/ |  
├── media/ | ├── stylesheets/ | ├── style.css | ├── faq.css └── .env
```

Oversikt over tabeller

Tabell 1:

- Navn: user
- Beskrivelse: Innholder en brukers info om email, brukernavn og et hashet og saltet passord.

Tabell 2:

- Navn: boardgame
- Beskrivelse: Inneholder brettspillet navn, hvilket år det kom ut, de som lagde det, de som publiserte det og en beskrivelse.

Tabell 3:

- Navn: role
- Beskrivelse: Inneholder forskjellige roller som brukere kan ha. Her må jeg kjøre en manuell Insert, eller lage en funksjon i Python-filen.

Tabell 4:

- Navn: question
- Beskrivelse: Inneholder spørsmål om nettsiden fra brukere. Den har derfor fremmednøkkel til brukere sin ID.

Tabellstruktur i Databasen

Videre ser du strukturen på kommandoene brukt til å skape tabellene. Hvis du vil ha en mer grafisk fremstilling av tabellene, kan du sjekke ut [tabellstruktur.md](#) som du finner i dokumentasjonsmappen.

```
-- Rolletabell (nr. 3)  
CREATE TABLE role (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(20)  
);  
-- Innhold til rolletabell  
INSERT INTO role (name) VALUES ("admin"), ("editor"), ("user");  
  
-- Rolletabellens innhold ser dermed slik ut:
```

```
+-----+-----+
| id | name |
+-----+-----+
| 1 | admin |
| 2 | editor |
| 3 | user |
+-----+-----+

-- Brukertabell (nr. 1)
CREATE TABLE `user` (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(255) NOT NULL UNIQUE,
  email VARCHAR(255) NOT NULL UNIQUE,
  password CHAR(60) NOT NULL,
  role_id INT, FOREIGN KEY (role_id) REFERENCES role(id) DEFAULT 3,
  active INT
);

-- Brettspilltabell (nr. 2)
CREATE TABLE boardgame (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  year_published INT,
  creator VARCHAR(255),
  publisher VARCHAR(255),
  img_filename VARCHAR(255),
  description TEXT CHARACTER SET utf8mb4
);

-- Spørsmålstabell (nr. 4)
CREATE TABLE question (
  id INT AUTO_INCREMENT PRIMARY KEY,
  question TEXT NOT NULL,
  created_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  user_id INT NOT NULL,
  FOREIGN KEY (user_id) REFERENCES `user`(id)
);
```

Hvordan sette opp dette systemet

Før du starter må du ha installert disse på systemet (dependencies):

- git
- python

Deretter kan du starte ved å klonе prosjektet:

```
git clone https://github.com/sivertmh/boardgame-site.git
```

Så installere nødvendige pakker for appen (Det er lurt å gjøre dette i et **venv** i python):

```
pip install -r requirements.txt
```

Nå kan du kjøre prosjektet lokalt fra terminalen:

```
python -m flask run
```

Du kan også kjøre med Waitress (kan da nås av andre på LAN):

```
# 0.0.0.0 gjør den tilgjengelig utover LAN-et med ip-adressen til systemet den
kjøres på
waitress-serve --host 0.0.0.0 app:app

# Eller, mer eksplisitt:
waitress-serve --listen 0.0.0.0:8080 app:app
```

Utenom server/database, er dette alt du trenger for grunnleggende bruk/test av Flask-appen. Uten kobling til database vises ikke brettspill og login vil ikke fungere. Hvis du endrer databasekoblingen til en db du har tilgang til, vil du kunne kjøre *app.py* og tabeller vil opprettes.

Hvis du får tilgang til database-filen kan du kjøre denne kommandoen for å importere databasen inn i Mariadb (Windows):

```
Get-Content "[databasefil]" | mariadb -u [brukernavn] -p [database]
```

Tilsvarende på linux vil være mye enklere:

```
mariadb -u [brukernavn] -p [database] < [databasefil]
```

Kilder:

Du finner kilder i dokumentet [klideliste.md](#) som er i mappen *dokumentasjon*.